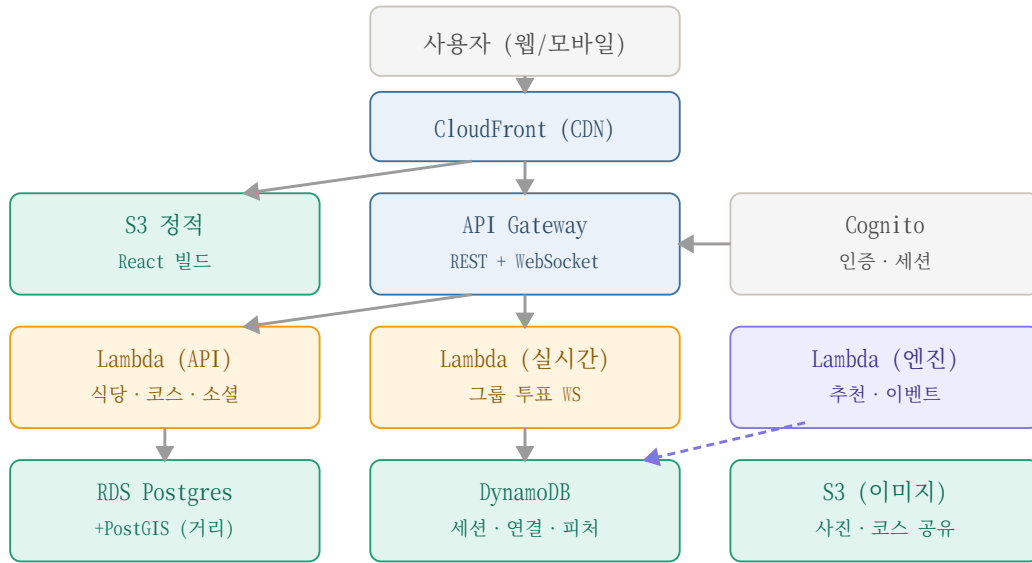


Lunchie × Munchie — AWS 아키텍처 설계서

서버리스 우선 · 비용 최적화 · 2026-06-15

실시간 그룹 투표 + 위치 기반 + 소셜 + 추천 엔진이라는 특성에 맞춰, 학생/팀 프로젝트의 비용을 0에 가깝게 유지하는 서버리스 우선(scale-to-zero) 구성으로 설계한다.

1. 아키텍처



비동기: Lambda → EventBridge/SQS → 활동 로깅 · 추천 피쳐 적재

운영: CloudWatch · Secrets Manager · IAM · AWS Budgets

[그림 1] 서버리스 우선 AWS 구성 — 엣지(파랑) · 컴퓨트(노랑) · 데이터(청록) · 엔진(보라) · 인증/운영(회색)

2. 구성요소

영역	서비스	역할 / 선택 이유
프론트	S3 + CloudFront	React 정적 호스팅 · CDN (Vercel 유지도 가능, 거의 무료)
API	API Gateway + Lambda	REST API; 트래픽 0이면 비용 0
실시간	API GW WebSocket + Lambda + DynamoDB	그룹 투표/스와이프 동기화 (서버 상주 없음)
DB	RDS Postgres + PostGIS	식당 · 코스 · 유저 + 반경 거리 쿼리
인증	Cognito	로그인 · 세션 (5만 MAU까지 무료)
스토리지	S3	사진 · 코스 공유 이미지 (presigned 업로드)
엔진/비동기	Lambda + EventBridge/SQS	활동 로깅 → 추천 피쳐 적재
운영	CloudWatch · Secrets Manager · IAM	모니터 · 시크릿 · 최소권한

3. 예상 버짓 (서울 ap-northeast-2)

① MVP / 데모 (저트래픽, 수백 유저)

항목	월 비용
S3 + CloudFront	\$1 ~ 5
API Gateway + Lambda	\$0 ~ 5 (월 100만 요청 무료)
RDS Postgres t4g.micro	\$12 ~ 15 (1년차 프리티어 \$0)
DynamoDB (온디맨드)	\$0 ~ 2
Cognito	\$0
S3 이미지 외	\$1 ~ 4
합계	약 \$15 ~ 35 / 월

DB를 Supabase 무료 티어로 유지하고 컴퓨트만 AWS로 올리면 월 \$5 ~ 10까지 내려간다. 초기엔 이 하이브리드를 권장.

② 성장기 (1k ~ 10k MAU)

항목	월 비용
RDS small / Aurora	\$50 ~ 100
Lambda + API Gateway	\$10 ~ 30
ElastiCache (Redis) 캐시 · 리더보드	\$15 ~ 30
CloudFront / S3 / 전송	\$10 ~ 30
DynamoDB	\$5 ~ 20
합계	약 \$100 ~ 200 / 월

③ ML 엔진 본격화 (SageMaker 학습 · 추론, ECS) → 필요할 때만 +\$150 ~ 400 / 월

4. 관리 유의사항 (예산 합정 위주)

- AWS Budgets + 결제 알림을 1일차에 설정 (예: \$30 초과 시 메일) — 사고 예방의 핵심
- NAT Gateway 합정: 상주 ~ \$32/월 + 데이터 요금 → Lambda를 VPC 밖에 두거나 VPC 엔드포인트 사용
- Aurora Serverless v2 최소 용량 (~\$44/월) — 저트래픽엔 RDS t4g.micro가 훨씬 싼
- 프리티어 12개월 만료 주의 — 1년 뒤 과금 시작, 캘린더 표시
- 상주 리소스 최소화 — 유틸 RDS 자동 중지, 안 쓰는 데모 환경 tear-down
- IaC(CDK/Terraform)로 생성 · 삭제 일원화 → orphan 리소스 과금 방지
- 보안: RDS private subnet + 보안그룹 최소화, 자격증명은 Secrets Manager(깃 금지), IAM 최소권한
- dev는 단일 AZ(저비용), prod DB만 Multi-AZ(가용성, 비용 2배)
- 리전 1개 · cost allocation 태깅 · egress 비용 의식(CloudFront로 완화)
- 학생이면 AWS Educate / GitHub Student Pack 크레딧 신청 — 사실상 무료 데모

5. 이행 순서

- 지금: 프론트(Vercel or S3+CloudFront) + Supabase Postgres 유지 → 비용 거의 0

- AWS 1차: 실시간 투표(API GW WebSocket+Lambda+DynamoDB) + 이미지(S3)를 AWS로
- AWS 2차: API를 Lambda로, DB를 RDS+PostGIS로 마이그레이션
- 엔진: EventBridge 이벤트 로깅 → 추천 Lambda → (나중에) SageMaker